

Deep Learning

15 Self-Attention & Transformers - II

Computational Complexity

- ① The attention layer has N i/p tokens and N o/p tokens, each of D dimensions

Computational Complexity

- ① The attention layer has N i/p tokens and N o/p tokens, each of D dimensions
- ② The overall dimension of i/p (or, o/p) is $N.D$

Computational Complexity

- ① The attention layer has N i/p tokens and N o/p tokens, each of D dimensions
- ② The overall dimension of i/p (or, o/p) is $N.D$
- ③ If we used a single fully-connected layer

Computational Complexity

- ① The attention layer has N i/p tokens and N o/p tokens, each of D dimensions
- ② The overall dimension of i/p (or, o/p) is $N.D$
- ③ If we used a single fully-connected layer
 - Will have $\mathcal{O}(N^2.D^2)$ independent parameters

Computational Complexity

- ① The attention layer has N i/p tokens and N o/p tokens, each of D dimensions
- ② The overall dimension of i/p (or, o/p) is $N.D$
- ③ If we used a single fully-connected layer
 - Will have $\mathcal{O}(N^2.D^2)$ independent parameters
 - Computational cost for one forward pass: $\mathcal{O}(N^2.D^2)$

Computational Complexity

- ① In an attention layer, $W^{(Q)}$, $W^{(K)}$, $W^{(V)}$ are shared across all the i/p tokens

Computational Complexity

- ① In an attention layer, $W^{(Q)}$, $W^{(K)}$, $W^{(V)}$ are shared across all the i/p tokens
- ② $\rightarrow$ No. of independent parameters is $\mathcal{O}(D^2)$ ($D_v \approx D_k \approx D$)

Computational Complexity

- ① In an attention layer, $W^{(Q)}, W^{(K)}, W^{(V)}$ are shared across all the i/p tokens
- ② $\rightarrow$ No. of independent parameters is $\mathcal{O}(D^2)$ ($D_v \approx D_k \approx D$)
- ③ For the N i/p tokens

Computational Complexity

- ① In an attention layer, $W^{(Q)}, W^{(K)}, W^{(V)}$ are shared across all the i/p tokens
- ② $\rightarrow$ No. of independent parameters is $\mathcal{O}(D^2)$ ($D_v \approx D_k \approx D$)
- ③ For the N i/p tokens
 - No. of computations required for computing the dot products in self-attention layer $\mathcal{O}(N^2.D)$

Computational Complexity

- ① In an attention layer, $W^{(Q)}, W^{(K)}, W^{(V)}$ are shared across all the i/p tokens
- ② $\rightarrow$ No. of independent parameters is $\mathcal{O}(D^2)$ ($D_v \approx D_k \approx D$)
- ③ For the N i/p tokens
 - No. of computations required for computing the dot products in self-attention layer $\mathcal{O}(N^2.D)$
- ④ Subsequent Neural Network layer has D inputs and D outputs $\rightarrow$ parameter = $\mathcal{O}(D^2)$ and computational cost of $\mathcal{O}(N.D^2)$

Positional Encoding

- ① The weights ($W^{(Q)}$, $W^{(K)}$, $W^{(V)}$) are shared across the i/p tokens $\rightarrow$ Transformer is permutation invariant

Positional Encoding

- ① The weights ($W^{(Q)}$, $W^{(K)}$, $W^{(V)}$) are shared across the i/p tokens $\rightarrow$ Transformer is permutation invariant
- ② Strong limitation to processing the sequential data

Positional Encoding

- ① The weights ($W^{(Q)}$, $W^{(K)}$, $W^{(V)}$) are shared across the i/p tokens $\rightarrow$ Transformer is permutation invariant
- ② Strong limitation to processing the sequential data
- ③ CSK plays better, not MI vs. MI plays better, not CSK

Positional Encoding

- ① The weights ($W^{(Q)}$, $W^{(K)}$, $W^{(V)}$) are shared across the i/p tokens $\rightarrow$ Transformer is permutation invariant
- ② Strong limitation to processing the sequential data
- ③ CSK plays better, not MI vs. MI plays better, not CSK
- ④ We need a way to inject the order information

Positional Encoding

- ① Without disturbing the design of the transformer, we may encode the position information (r_n) along with the data (tokens) (x_n)

Positional Encoding

- ① Without disturbing the design of the transformer, we may encode the position information (r_n) along with the data (tokens) (x_n)
- ② Obvious way is to concatenate

Positional Encoding

- ① Without disturbing the design of the transformer, we may encode the position information (r_n) along with the data (tokens) (x_n)
- ② Obvious way is to concatenate
 - Increased dimensions and computations

Positional Encoding

- ① Without disturbing the design of the transformer, we may encode the position information (r_n) along with the data (tokens) (x_n)
- ② Obvious way is to concatenate
 - Increased dimensions and computations
- ③ Instead, add them $\tilde{x}_n = x_n + r_n$

Positional Encoding

- ① Would it not corrupt the data?

Positional Encoding

- ① Would it not corrupt the data?
 - High dimensionality keeps them separate

Positional Encoding

- ① Would it not corrupt the data?
 - High dimensionality keeps them separate
 - Skip connections retain the r_n across the layers

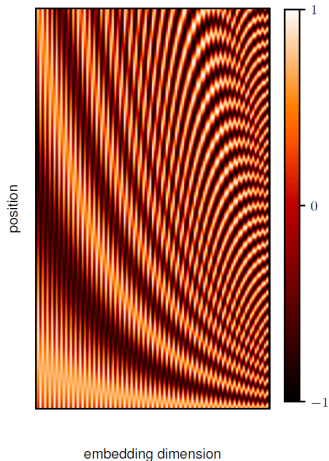
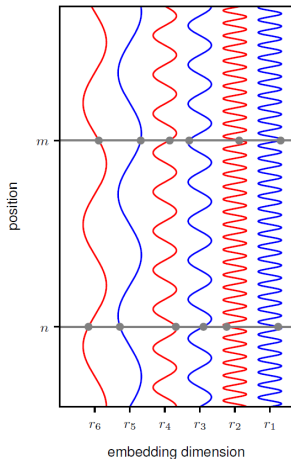
Positional Encoding

An ideal position encoding should provide unique representation for each position, should be bounded, should generalize to longer sequences

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even,} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd.} \end{cases}$$

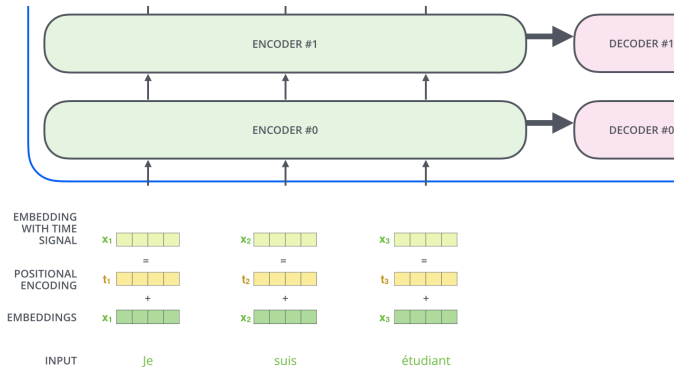
The Bishop's book

Positional Encoding



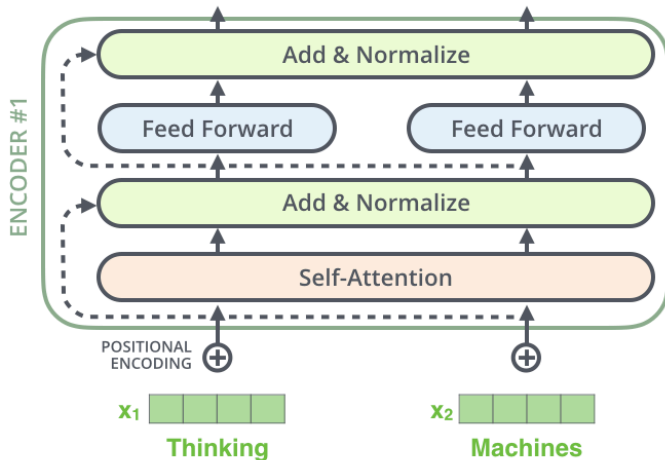
The Bishop's book

Positional Encoding



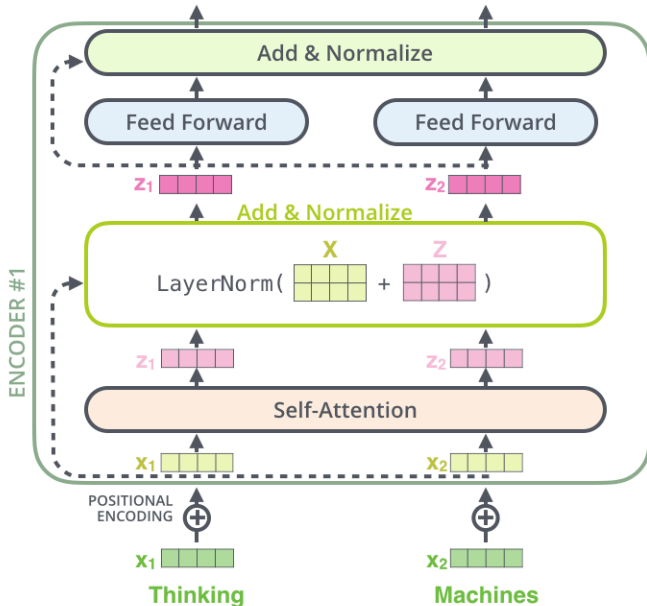
Credits: Jay Alamm

Residuals in the Encoder

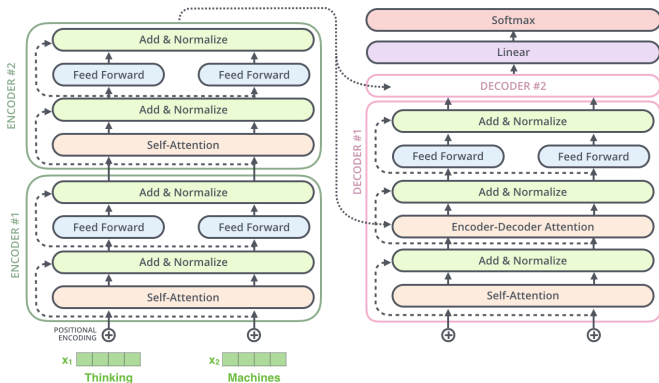


Credits: Jay Alammar

Residuals in the Encoder



Transformer-Decoder



Credits: Jay Alammar

Transformer-Decoder

- ① Self-attention here works in a slightly different way → masks the future positions

Transformer-Decoder

- ① Self-attention here works in a slightly different way → masks the future positions
- ② Uses the top encoder's K and V vectors for its' encoder-decoder (cross) attention

Transformer-Decoder

- ① Self-attention here works in a slightly different way → masks the future positions
- ② Uses the top encoder's K and V vectors for its' encoder-decoder (cross) attention
- ③ Encoder-decoder attention layer borrows the queries from the layer below it